

链甲 AgentArmor

Multi-Agent Shared Write 安全防御

阶段性设计与测试报告

2026 年 6 月 28 日

目录

摘要	1
第一章 介绍	2
1.1 研究背景	2
1.2 问题定义	2
1.3 主要贡献	2
第二章 背景与威胁模型	4
2.1 Multi-Agent Shared Write 场景	4
2.2 攻击者能力	4
2.3 攻击目标	4
2.4 攻击面分析	4
2.5 安全不变量	5
第三章 系统设计	6
3.1 设计目标	6
3.2 总体架构	6
3.3 核心数据抽象	6
3.4 写入、检索与执行三重边界	7
第四章 攻击分类与防御算法	8
4.1 符号说明	8
4.2 端到端攻击流程	8
4.3 共享写入防御算法	9
4.4 风险评分算法	9
4.5 检索审计算法	10
4.6 执行仲裁算法	11
4.7 撤销与影响分析算法	12
第五章 实验与评估	13
5.1 研究问题	13
5.2 数据集构造	13
5.3 评价指标	13
5.4 RQ1: 节点单独效果	14
5.5 RQ2 与 RQ3: 消融实验	14

5.6	RQ4: 技术选型	15
5.7	RQ5: 基线对比	15
5.8	RQ6: 端到端验证	15
5.9	延迟测试	16
第六章	总结与讨论	17
6.1	阶段结论	17
6.2	局限性	17
6.3	未来工作	17

摘要

多 Agent 系统正在从单轮问答扩展到长期协作、共享记忆和自动工具调用。与传统单 Agent 场景相比，多个 Agent 共同读写同一份记忆时，会产生一种新的信任传播风险：低信任外部内容经过某个 Agent 的读取、摘要或改写后，被写入共享记忆；后续 Agent 再将这段记忆当作可信事实或执行依据，从而触发越权工具调用、数据外传或长期污染。

本文针对该类风险定义 Type 5: Multi-Agent Shared Write 场景，并提出 MASW 防御框架。框架围绕“不可信内容不得因 Agent 改写或写入而自动提升为可信记忆”这一安全不变量，将防御链拆分为输入标记、候选事实抽取、风险过滤、溯源写入网关、检索审计、执行仲裁和事后撤销七个阶段。与单纯文本分类器不同，MASW 的核心目标不是只判断某句话是否危险，而是在共享记忆的写入、检索和执行三个关键边界上阻断攻击链。

为了验证系统有效性，本文构建了最小可用测试集，包含 prompt injection、tool misuse、memory poisoning、agent hijacking 四类攻击样本各 20 条，并加入 45 条良性样本；在技术选型和基线对比中进一步扩展到 145 条样本。实验包括单元测试、消融实验、节点效果分析、技术选型、基线对比、延迟测试和 8 个端到端攻击场景。结果显示，MASW 在当前数据集上阻断了全部攻击样本的共享写入；在端到端验证中，脆弱系统发生 8/8 次危险执行，而 MASW 将危险执行降为 0/8。

关键词：多 Agent 安全；共享记忆；提示注入；记忆投毒；工具调用；溯源验证

第一章 介绍

1.1 研究背景

多 Agent 系统通常由多个具备不同任务角色的 Agent 共同完成复杂流程。例如，一个 Agent 负责检索网页，一个 Agent 负责总结知识，一个 Agent 负责生成执行计划，另一个 Agent 负责调用邮件、数据库或代码仓库工具。为了提升协作效率，这类系统往往会引入共享对话日志、crew memory、共享知识库或 RAG memory store，使不同 Agent 可以复用彼此产生的中间结果。

共享记忆提升了效率，也放大了安全风险。单 Agent 场景中的提示注入通常只影响当前对话；而在 Multi-Agent Shared Write 场景中，一次恶意输入可能被写入长期共享状态，并在未来的正常任务中被其他 Agent 召回。这使攻击从“即时影响一次回答”升级为“跨 Agent、跨任务、可持久化触发”的系统风险。

1.2 问题定义

本文研究的系统流程如下：

External Data $\rightarrow$ Agent A $\rightarrow$ Shared Memory $\rightarrow$ Agent B/C $\rightarrow$ Tool Action

在脆弱系统中，Agent A 只要拥有写入权限，其输出就可能直接进入共享记忆；Agent B/C 在后续任务中检索到该记忆后，又可能把它当成可信指令或事实使用。攻击者不需要直接调用高权限工具，只需要控制网页、工具返回、邮件或文档中的一段文本，即可通过低权限 Agent 完成间接影响。

因此，本文关注的核心问题是：

在多个 Agent 平等写入共享记忆的系统中，如何防止低信任外部内容被自动提升为高信任共享记忆，并进一步诱导后续 Agent 执行危险操作？

1.3 主要贡献

本文当前阶段的主要贡献包括：

1. 提出 Multi-Agent Shared Write 场景下的信任提升风险模型，明确指出共享记忆的危险不在于“存储文本”，而在于“文本被后续 Agent 当作可信上下文使用”。
2. 构造 MASW-POISON-EXEC 端到端攻击链，复现外部注入、共享记忆污染、后续检索和危险工具执行的完整流程。
3. 设计 MASW 分层防御框架，在写入、检索和执行三个边界分别设置溯源验证、信任审计和动作仲裁。

4. 构建最小可用评测数据集和端到端验证场景，使用消融实验、节点效果、基线对比和延迟测试验证当前设计。

第二章 背景与威胁模型

2.1 Multi-Agent Shared Write 场景

Multi-Agent Shared Write 指多个 Agent 均可向共享记忆写入内容，且写入内容会在后续任务中被其他 Agent 检索和使用。其典型特征包括：

1. **写入主体多样**：任意 Agent 都可能成为共享记忆的写入者。
2. **内容来源复杂**：写入内容可能来自用户、网页、搜索结果、工具返回、文件或模型总结。
3. **使用时机延迟**：污染内容不一定立即触发危害，可能在未来任务中被召回。
4. **权限不对称**：写入内容的 Agent 权限可能较低，读取内容并执行工具的 Agent 权限可能较高。

这类系统的隐含假设是：共享记忆中的内容具有较高可信度。但该假设在外部内容可被写入时不成立。

2.2 攻击者能力

本文假设攻击者可以控制至少一个外部内容源，例如网页、搜索结果、工具返回、文档片段、邮件正文或 OCR 文本。攻击者能够诱导低权限 Agent A 读取该内容，并构造自然语言、中英文混合、伪装业务规则、伪装知识库同步流程等 payload。

攻击者不能直接修改系统策略，不能直接调用高权限工具，也不能修改 MASW 防御代码。换言之，本文研究的不是系统被完全攻破后的情况，而是外部不可信内容经由正常 Agent 流程进入共享记忆后造成的间接攻击。

2.3 攻击目标

攻击者目标可以分为四类：

1. **完整性破坏**：将恶意规则、伪政策或伪任务规范写入共享记忆。
2. **机密性破坏**：诱导后续 Agent 外传上下文、摘要、密钥或内部数据。
3. **权限绕过**：让高权限 Agent 调用 `email.send`、`database.write`、`repo.commit`、`deploy.production`、`secret.read` 等工具。
4. **持久化影响**：使污染内容跨任务、跨 Agent 留存，并在未来被正常查询触发。

2.4 攻击面分析

表 2.1: Multi-Agent Shared Write 攻击面分析

阶段	系统状态	潜在威胁	失败根因
外部输入	网页、文件、工具返回、邮件	隐藏指令、伪装规范、跨语言注入	内容缺少来源、信任和污点标签
Agent 处理	摘要、抽取、改写	指令被改写成事实	生成文本被误当作验证事实
共享写入	候选内容进入 shared memory	自动信任提升、污染持久化	写入权限扁平，缺少网关
记忆检索	后续 Agent 召回记忆	污染内容被当成上下文	检索只看相关性，不看风险
工具执行	Agent 生成 action proposal	数据外传、越权写入、部署操作	Agent 输出被直接当作授权
事后处置	污染已传播	派生内容无法清理	缺少溯源图和影响分析

2.5 安全不变量

MASW 的核心安全不变量为：

No Automatic Trust Promotion

即：不可信外部内容不能仅因为经过 Agent 摘要、改写或写入，就自动变成可信共享记忆。

设内容项为 x ，来源信任为 $T_{\text{src}}(x)$ ，写入者信任为 $T_{\text{writer}}(a)$ ，共享记忆信任为 $T_{\text{mem}}(x)$ 。脆弱系统常见的错误规则为：

$$T_{\text{mem}}(x) = T_{\text{writer}}(a)$$

该规则忽略了内容来源，使得低信任外部数据可以借助高权限或可写 Agent 完成信任漂白。本文将其改写为：

$$T_{\text{mem}}(x) = \begin{cases} \text{promote}(x), & V(x) = 1 \wedge R(x) \leq \tau_w, \\ T_{\text{src}}(x), & \text{otherwise,} \end{cases}$$

其中 $V(x)$ 表示溯源验证是否通过， $R(x)$ 表示风险分数， τ_w 表示共享写入阈值。

第三章 系统设计

3.1 设计目标

MASW 的设计目标不是替代所有 Agent 框架，也不是对所有文本做统一拦截，而是在 Multi-Agent Shared Write 场景中建立清晰的安全边界。系统需要满足以下要求：

1. 外部内容进入系统时必须带有来源、信任和污点标签。
2. Agent 生成的中间结果只能作为候选事实，不能直接成为共享可信记忆。
3. 共享记忆写入必须经过独立验证，不能由写入 Agent 自行决定可信度。
4. 检索结果只能作为证据，不能天然作为执行指令。
5. 高影响工具调用必须经过动作仲裁和权限检查。
6. 污染确认后，系统能够追踪并撤销派生记忆。

3.2 总体架构

MASW 将完整防御链划分为七个阶段：

1. **D1 输入标记**：为外部输入绑定 source、channel、trust、taint 和 scope。
2. **D2 候选事实抽取**：将 Agent A 的输出限制为候选事实，不允许直接写入共享记忆。
3. **D3 风险过滤**：检测提示注入、工具滥用、外传、记忆投毒和 Agent 劫持信号。
4. **D4 溯源写入网关**：依据证据、来源、冲突和风险决定是否允许写入。
5. **D5 检索审计**：检索时同时考虑相关性、信任等级、作用域和风险。
6. **D6 执行仲裁**：高影响动作必须与用户目标、权限和上下文一致。
7. **D7 撤销与影响分析**：基于溯源图追踪污染节点及其派生内容。

这七个阶段对应攻击链中的不同断点：D1-D4 控制污染是否进入共享记忆，D5-D6 控制污染是否能影响执行，D7 控制污染发生后的扩散影响。

3.3 核心数据抽象

为了避免自然语言内容在不同阶段被误用，MASW 将系统状态分为四类对象：

$$X = \langle text, source, channel, trust, taint, scope \rangle$$

$$C = \langle claim, evidence, parent, taint, risk_hint \rangle$$

$$M = \langle claim, provenance, trust, risk, scope, parent_ids \rangle$$

$$A = \langle tool, args, user_goal, context, impact \rangle$$

其中 X 表示带标签输入， C 表示候选事实， M 表示共享记忆记录， A 表示工具动作提案。该抽象的作用是把“文本内容”和“可信状态”拆开：同一段文本在不同对象中具有不同权限，只有通过 D4 的候选事实才能转化为共享记忆。

3.4 写入、检索与执行三重边界

MASW 的关键不是某个单点检测器，而是三重边界：

1. **写入边界**：外部内容无法绕过 D3/D4 直接进入共享记忆。
2. **检索边界**：低信任或跨作用域记忆无法被当作高信任上下文使用。
3. **执行边界**：检索到的记忆不能自动授权高影响工具调用。

因此，即使某一层被绕过，后续层仍可阻断攻击链。

第四章 攻击分类与防御算法

4.1 符号说明

表 4.1: 伪代码符号表

符号	含义	符号	含义
$\mathcal{X}$	带标签外部输入集合	$\mathcal{C}$	候选事实集合
$\mathcal{M}$	共享记忆集合	$\mathcal{Q}$	隔离队列
$\mathcal{A}$	动作提案集合	$\mathcal{G}$	记忆溯源图
$T(x)$	输入或记忆的信任分数	$R(c)$	候选事实风险分数
τ_3	D3 风险过滤阈值	τ_4	D4 写入阈值
$E(c)$	证据验证谓词	$S(c)$	来源验证谓词
$K(c)$	冲突检测谓词	$P(a)$	动作权限谓词

4.2 端到端攻击流程

MASW-POISON-EXEC 攻击的目标是让恶意外部内容跨过 Agent A，进入共享记忆，并在 Agent B 的正常任务中被召回后触发危险工具。该攻击过程可形式化为 Algorithm 1。

Algorithm 1: MASW-POISON-EXEC 攻击复现流程

Input : 外部内容源 d , 注入载荷 p , 低权限 Agent A_1 , 高权限 Agent A_2 , 共享记忆 $\mathcal{M}$, 目标查询 q_t

Output : 危险动作 a^* 或攻击失败标记 $\perp$

```

1  $d' \leftarrow \text{Inject}(d, p);$                                 ▷ 攻击者把伪规则写入网页、工具返回或文档
2  $s \leftarrow A_1.\text{ReadAndSummarize}(d');$                 ▷ 低权限 Agent 将载荷改写为经验或规范
3 if  $\text{VulnerableWritePolicy}(s) = \text{true}$  then
4    $\mathcal{M} \leftarrow \mathcal{M} \cup \{s\};$                         ▷ 污染内容进入共享记忆
5  $\mathcal{R} \leftarrow A_2.\text{Retrieve}(\mathcal{M}, q_t);$ 
6  $a^* \leftarrow A_2.\text{PlanAction}(q_t, \mathcal{R});$ 
7 if  $a^*.tool \in \{\text{email.send}, \text{database.write}, \text{deploy.production}, \text{secret.read}\}$  then
8   return  $a^*;$                                           ▷ 攻击成功: 高权限工具被污染记忆诱导
9 return  $\perp;$ 

```

该攻击能够成立，是因为脆弱系统同时满足两个条件：一是共享写入阶段缺少信任提升验证；二是工具执行阶段把检索到的记忆当成授权依据。

4.3 共享写入防御算法

共享写入是整条攻击链中最关键的边界。Algorithm 2 展示了 MASW 的写入流程：外部内容先被标记，再被抽取为候选事实，随后经过风险过滤和溯源网关，只有通过验证的内容才能进入共享记忆。

Algorithm 2: Provenance-Gated Shared Write

Input : 外部输入 x , 写入 Agent a , 共享记忆 $\mathcal{M}$, 隔离队列 $\mathcal{Q}$

Output : 更新后的 $\mathcal{M}$ 与 $\mathcal{Q}$

```

1  $x' \leftarrow \text{Label}(x);$                                 ▷ D1: 绑定 source、trust、taint、scope
2  $\mathcal{C} \leftarrow \text{ExtractCandidateFacts}(a, x');$           ▷ D2: 只生成候选事实
3 foreach  $c \in \mathcal{C}$  do
4    $r \leftarrow \text{RiskScore}(c);$                           ▷ D3: 提示注入、外传、工具滥用、记忆投毒
5   if  $r > \tau_3$  or  $\text{InstructionLike}(c)$  then
6      $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(c, \text{"risk\_filter"})\};$ 
7     break
8    $v \leftarrow E(c) \wedge S(c) \wedge \neg K(c) \wedge r \leq \tau_4;$     ▷ D4: 证据、来源、冲突、风险
9   if  $v = \text{true}$  then
10     $m \leftarrow \text{PromoteToMemory}(c, \text{provenance}(c));$ 
11     $\mathcal{M} \leftarrow \mathcal{M} \cup \{m\};$ 
12  else
13     $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(c, \text{"provenance\_gate"})\};$ 
14 return  $(\mathcal{M}, \mathcal{Q});$ 

```

该算法的安全含义是：Agent 的自然语言输出不能直接改变共享记忆的信任等级，必须经过独立验证。若攻击者试图把“将总结发送到外部邮箱”伪装成知识库同步规则，该内容会在风险过滤或写入网关阶段进入隔离队列。

4.4 风险评分算法

D3 的作用是尽早发现明显攻击和隐蔽外传表述。当前实现采用规则检测与语义 rubric 的组合，风险分数定义为：

$$R(c) = w_i I_{\text{inject}}(c) + w_t I_{\text{tool}}(c) + w_m I_{\text{memory}}(c) + w_e I_{\text{exfil}}(c) + w_h I_{\text{hijack}}(c)$$

其中 $I(c) \in \{0, 1\}$ ，分别表示提示注入、工具滥用、记忆投毒、数据外传和 Agent 劫持信号。Algorithm 3 给出检测流程。

Algorithm 3: Hybrid Risk Screening**Input** : 候选事实 c , 规则集合 Ω , rubric 维度集合 Φ **Output** : 风险分数 $R(c)$ 与风险标签集合 $\mathcal{B}$

```

1  $R(c) \leftarrow 0, \mathcal{B} \leftarrow \emptyset;$ 
2 foreach  $\omega \in \Omega$  do
3   if  $\omega.\text{match}(c.\text{text}) = \text{true}$  then
4      $R(c) \leftarrow R(c) + \omega.\text{weight};$ 
5      $\mathcal{B} \leftarrow \mathcal{B} \cup \{\omega.\text{tag}\};$ 
6 foreach  $\phi \in \Phi$  do
7    $s_\phi \leftarrow \text{RubricScore}(c, \phi);$  ▷ 检查同步伪装、外部路由、工具副作用等语义信号
8   if  $s_\phi > \phi.\text{threshold}$  then
9      $R(c) \leftarrow R(c) + \phi.\text{weight};$ 
10     $\mathcal{B} \leftarrow \mathcal{B} \cup \{\phi.\text{tag}\};$ 
11 if  $\text{ExternalRecipient}(c) \wedge \text{ActionVerb}(c)$  then
12    $R(c) \leftarrow R(c) + w_e, \mathcal{B} \leftarrow \mathcal{B} \cup \{\text{exfiltration}\};$ 
13 return  $(R(c), \mathcal{B});$ 

```

基础实验显示，纯规则检测对显式攻击足够有效，但在“知识库同步”“团队归档”“自动推送”等隐蔽外传表达上存在漏报。因此 D3 使用 hybrid 策略；D4 写入网关则保持更保守的规则检测，以减少良性内部同步被误伤。

4.5 检索审计算法

即使污染内容绕过写入阶段，也不应在检索阶段被无条件交给后续 Agent。D5 将检索排序由单一相关性扩展为信任感知评分：

$$S(m, q) = \text{sim}(m, q) + \alpha T(m) - \beta R(m) - \gamma I_{\text{scope}}(m, q)$$

其中 $\text{sim}(m, q)$ 是相关性， $T(m)$ 是记忆信任， $R(m)$ 是风险分数， I_{scope} 表示作用域不匹配。

Algorithm 4: Trust-Aware Retrieval Audit**Input** : 目标查询 q , 共享记忆 $\mathcal{M}$, 最小信任阈值 τ_T , 最大风险阈值 τ_R **Output** : 可进入上下文的记忆集合 $\mathcal{R}$

```

1  $\mathcal{R} \leftarrow \emptyset$ ;
2 foreach  $m \in \mathcal{M}$  do
3   if ScopeMismatch( $m, q$ ) = true then
4     continue;                                ▷ 跨任务或跨租户内容不进入上下文
5   if  $T(m) < \tau_T$  or  $R(m) > \tau_R$  then
6     AuditLog( $m, q$ , "filtered");
7     continue;
8    $score \leftarrow \text{sim}(m, q) + \alpha T(m) - \beta R(m)$ ;
9    $\mathcal{R} \leftarrow \mathcal{R} \cup \{(m, score, \text{evidence\_only})\}$ ;
10  $\mathcal{R} \leftarrow \text{TopK}(\mathcal{R})$ ;
11 return  $\mathcal{R}$ ;

```

该算法保证检索结果的身份是 evidence, 而不是 instruction。后续 Agent 可以参考其事实内容, 但不能把它当作工具授权规则。

4.6 执行仲裁算法

D6 处理的是混淆代理风险。即使某条污染记忆进入上下文, 只要它诱导高影响工具调用, 动作仍必须通过独立仲裁。设动作 a 的允许条件为:

$$\text{Allow}(a) = U(a) \wedge P(a) \wedge \neg H(a) \wedge \neg X(a)$$

其中 $U(a)$ 表示与用户目标一致, $P(a)$ 表示权限满足, $H(a)$ 表示高影响动作, $X(a)$ 表示外传或危险副作用。

Algorithm 5: Execution Mediation for High-Impact Tools**Input** : 动作提案 a , 用户目标 g , 检索上下文 $\mathcal{R}$, 工具策略 Π **Output** : 执行决策 $d \in \{\text{allow}, \text{deny}, \text{review}\}$

```

1 if  $\neg \text{GoalAligned}(a, g)$  then
2   return deny;                                ▷ 动作与用户任务不一致
3 if  $\neg \text{PermissionOK}(a, \Pi)$  then
4   return deny;                                ▷ 权限不足
5 if  $\text{ExternalSink}(a) \wedge \text{SensitiveContext}(\mathcal{R})$  then
6   return review;                                ▷ 存在数据外传风险
7 if  $a.\text{tool} \in$ 
   {email.send, database.write, repo.commit, deploy.production, secret.read}
   then
8   return review;                                ▷ 高影响工具必须人工确认
9 return allow;

```

该算法并不依赖 D3/D4 是否成功。只要动作具有高影响或外传风险，就不会因为某段共享记忆声称“这是团队规范”而自动执行。

4.7 撤销与影响分析算法

D7 将共享记忆视为有向图 $\mathcal{G} = (V, E)$ ，边 $u \rightarrow v$ 表示记忆 v 由 u 派生。污染节点 p 的影响集合为：

$$\text{Impact}(p) = \{v \in V \mid p \rightsquigarrow v\}$$

Algorithm 6: Revocation and Impact Analysis

Input : 溯源图 $\mathcal{G}$ ，污染记忆节点 p ，共享记忆 $\mathcal{M}$

Output : 撤销后的共享记忆 $\mathcal{M}'$ 与影响集合 $\mathcal{I}$

```

1  $\mathcal{I} \leftarrow \{p\}, W \leftarrow \{p\};$ 
2 while  $W \neq \emptyset$  do
3    $u \leftarrow \text{pop}(W);$ 
4   foreach  $v \in \text{Children}_{\mathcal{G}}(u)$  do
5     if  $v \notin \mathcal{I}$  then
6        $\mathcal{I} \leftarrow \mathcal{I} \cup \{v\}, W \leftarrow W \cup \{v\};$ 
7 foreach  $m \in \mathcal{I}$  do
8    $\mathcal{M} \leftarrow \mathcal{M} \setminus \{m\};$ 
9    $\text{AuditLog}(m, \text{"revoked"});$ 
10 return  $(\mathcal{M}, \mathcal{I});$ 

```

D7 不阻止首次攻击，但解决污染发现后的持续传播问题。没有 parent ids 和 provenance graph 时，系统只能删除单条记忆，无法判断哪些后续内容也被污染。

第五章 实验与评估

5.1 研究问题

本文通过六个研究问题评估 MASW：

- RQ1：每个防御节点能否防住对应攻击阶段？
- RQ2：系统能否阻断共享记忆污染？
- RQ3：组合防御是否优于单点防御？
- RQ4：D3/D4 的检测技术路线应如何选择？
- RQ5：与同类工具基线相比，MASW 是否更有效？
- RQ6：真实两轮攻击链能否被复现并阻断？

5.2 数据集构造

当前阶段先从最小可用规模开始构建数据集。基础集共 125 条，其中攻击样本 80 条，良性样本 45 条。技术选型和基线对比额外加入 20 条隐蔽同步、语义改写和边界良性样本，形成 145 条 hard set。

表 5.1: 数据集类别与覆盖内容

类别	数量	覆盖内容
Prompt Injection	20	忽略原指令、伪系统消息、隐藏规则、跨语言注入、上下文泄露。
Tool Misuse	20	邮件外发、数据库写入、仓库提交、生产部署、密钥读取。
Memory Poisoning	20	伪知识库规则、伪同步策略、覆盖安全策略、长期持久化规范。
Agent Hijacking	20	改变后续 Agent 目标、转发上下文、执行攻击者任务。
Benign	45	正常搜索总结、知识整理、内部同步、无危险副作用任务。

公开数据集如 PromptInject 和 JailbreakBench 可提供提示注入表达参考，但它们通常不包含共享记忆写入、检索、隔离、工具执行等标签。因此本项目采用“公开攻击模式参考 + 本系统场景重构”的方式构造样本。

5.3 评价指标

本文使用 Precision、Recall、F1 和 FPR 衡量拦截效果：

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

端到端实验额外记录 poisoned memory write 和 dangerous execution；性能实验记录 P50、P95、P99 延迟。

5.4 RQ1：节点单独效果

表 5.2: 节点单独效果

节点配置	对应阶段	Prec.	Rec.	F1	FPR	结论
D1 only	输入标记	0.6400	1.0000	0.7805	1.0000	不能独立拦截
D2 only	候选抽取	0.0000	0.0000	0.0000	0.0000	中间层
D3 only	风险过滤	1.0000	1.0000	1.0000	0.0000	基础集有效
D4 gate	写入网关	1.0000	1.0000	1.0000	0.0000	核心边界
D5 only	检索审计	1.0000	0.0625	0.1176	0.0000	后置防线
D6 only	执行仲裁	0.6400	1.0000	0.7805	1.0000	单独误伤高
D7	撤销处置	1.0000	1.0000	1.0000	0.0000	事故后收束

结果表明，D3 和 D4 是当前最直接的拦截边界；D1、D2 是前置结构化条件；D5、D6、D7 更偏向纵深防御和事故后收束，不能单独替代写入网关。

5.5 RQ2 与 RQ3：消融实验

表 5.3: Config-1 到 Config-5 消融实验

Config	启用阶段	Prec.	Rec.	F1	FPR
Config-1 Baseline	none	0.0000	0.0000	0.0000	0.0000
Config-2 D3 only	D2+D3	1.0000	1.0000	1.0000	0.0000
Config-3 D4 gate	D1+D2+D4	1.0000	1.0000	1.0000	0.0000
Config-4 D3+D4	D1-D4	1.0000	1.0000	1.0000	0.0000
Config-5 ALL	D1-D7	1.0000	1.0000	1.0000	0.0000

基础集上 D3/D4 已经饱和，因此严格意义上的 $F1_{ALL} > F1_{best}$ 不成立。该结果不能被夸大组合防御在所有指标上都具有数学协同，而应解释为：基础集难度不足以体现后置边界的贡献。端到端攻击链验证显示，当写入、检索和执行串联起来时，组合防御仍是必要的。

5.6 RQ4: 技术选型

表 5.4: 检测器技术选型结果

Config	Prec.	Rec.	F1	FPR	结论
D3_RULE	1.0000	0.8889	0.9412	0.0000	漏掉隐蔽表达
D3_HYBRID	1.0000	1.0000	1.0000	0.0000	推荐用于 D3
D4_GATE_RULE	1.0000	1.0000	1.0000	0.0000	推荐用于 D4
D4_GATE_HYBRID	0.9783	1.0000	0.9890	0.0364	写入误伤升高
D3 Hybrid + D4 Rule	1.0000	1.0000	1.0000	0.0000	当前推荐

结论是：D3 位于早期筛查阶段，应该提高召回，因此使用 hybrid；D4 位于共享写入最终边界，应该更保守，因此使用 rule-based verifier。

5.7 RQ5: 基线对比

表 5.5: 与基线工具的横向对比

Baseline	Status	Prec.	Rec.	F1	FPR
MASW_ALL	ok	1.0000	1.0000	1.0000	0.0000
PromptInject-style	ok	1.0000	0.3222	0.4874	0.0000
LLM-Guard-style	ok	1.0000	0.4333	0.6047	0.0000
Rebuff-style	ok	1.0000	0.4222	0.5938	0.0000
DeepSeek-v4-flash	skipped	–	–	–	–

三类本地工具基线主要判断文本是否像提示注入，缺少共享写入网关、检索审计和执行仲裁，因此对 Multi-Agent Shared Write 攻击链覆盖不足。DeepSeek 远程基线已预留接口，但当前报告不写入 API key，默认跳过远程调用。

5.8 RQ6: 端到端验证

端到端验证使用两轮任务：第一轮 Q_{inject} 使 Agent A 读取恶意外部内容并尝试写入共享记忆；第二轮 Q_{target} 使 Agent B 在正常任务中检索共享记忆并生成工具动作。

表 5.6: 端到端攻击链验证结果

指标	结果
Scenarios	8
Vulnerable path dangerous executions	8/8

指标	结果
MASW poisoned memory writes	0/8
MASW dangerous executions	0/8

8 个场景覆盖邮件外发、数据库写入、仓库提交、生产部署和密钥读取。脆弱系统失败的原因是共享记忆采用扁平信任模型；MASW 在 D4 写入网关和 D6 执行仲裁两个独立边界上切断了攻击链。

5.9 延迟测试

表 5.7: 延迟测试结果

Experiment	Config	P50 ms	P95 ms	P99 ms
Ablation	ALL	0.0427	0.0977	0.1020
Tech Selection	ALL recommended	0.0473	0.1121	0.1363
Baseline Compare	MASW_ALL	0.0485	0.1188	0.1411

当前实现为内存版和规则/伪 rubric 版，因此延迟处于亚毫秒级。后续接入真实 LLM judge、向量库和外部工具后，需要重新评估端到端延迟和调用成本。

第六章 总结与讨论

6.1 阶段结论

本文围绕 Multi-Agent Shared Write 场景，给出了从攻击链到防御链的完整阶段性验证。当前结论如下：

1. 共享记忆污染的本质是自动信任提升，而不是简单的单轮提示注入。
2. 写入网关是系统不可缺少的架构边界；没有 D4，系统无法证明 shared memory 中内容的可信来源。
3. D3 适合采用 hybrid 检测以提高隐蔽攻击召回，D4 应保持保守以控制良性同步误伤。
4. D5 和 D6 体现纵深防御价值：污染即使进入上下文，也不能自动成为工具授权。
5. 基础集上的指标饱和说明数据集仍需扩展，不能直接宣称系统已在真实世界完备有效。
6. 端到端验证已经复现攻击链，并证明 MASW 可将危险执行从 8/8 降为 0/8。

6.2 局限性

当前工作仍然存在局限。第一，数据集规模较小，样本主要来自公开攻击模式参考和人工重构，尚不能代表真实线上分布。第二，rubric 检测目前是确定性替身，还没有接入真实 LLM judge。第三，端到端 Agent 是最小复现系统，尚未接入 AutoGen、CrewAI 等真实多 Agent 框架。第四，当前延迟数据来自内存版实现，接入向量库、远程模型和人工审批后需要重新测量。

6.3 未来工作

下一阶段将围绕三个方向推进：一是扩展数据集，加入更多跨语言、长上下文、隐蔽同步和良性边界样本；二是补跑远程 LLM 基线，并在不泄露密钥的前提下评估成本、延迟和隐私风险；三是将 MASW 接入真实多 Agent 框架，验证共享记忆写入网关、检索审计和执行仲裁在真实工作流中的兼容性。